

transcription

Phase 0 Research — Transcription of Chapter 1

Feature: 001-workshop-curriculum-platform **Date measured:** 2026-08-31 **Host:** this machine (see [Machine baseline](#)) **Scope:** how to transcribe workshop/chapters/01/...Recording.mp4 accurately, verifiably and resumably, satisfying FR-001...FR-007, FR-029, FR-037, FR-038 and SC-001, SC-002, SC-003, SC-016.

Nothing was transcribed. No ASR job was started, no package was installed, no file under workshop/ was modified. Every number below is either a command output reproduced verbatim, or is explicitly marked **UNVERIFIED** with the reason it could not be measured here.

Reading guide

Findings are recorded as **Decision / Rationale / Alternatives considered**. Section 0 is measurement only — it carries no decision, but every decision after it depends on it.

A claim in this document is one of exactly three things, and they are never mixed:

Marker	Meaning
MEASURED	A command was run on this machine; its output is quoted.
UNVERIFIED	Not observed here. The reason is stated, and so is the command that would settle it.
DERIVED	

Marker	Meaning
	Arithmetic over MEASURED values. The arithmetic is shown.

Per §11.4.6, an UNVERIFIED item is never upgraded to a conclusion by confidence alone.

Machine baseline

```
$ nproc
8
$ lscpu | grep -E '^(Model name|CPU\(s\)|Thread|Core|Socket|CPU
max MHz|L3)'
CPU(s):                        8
Model name:                    11th Gen Intel(R)
Core(TM) i7-1165G7 @ 2.80GHz
Thread(s) per core:           2
Core(s) per socket:           4
Socket(s):                     1
CPU max MHz:                   4700.0000
L3 cache:                      12 MiB (1 instance)

$ grep -m1 '^flags' /proc/cpuinfo | tr ' ' '\n' | grep -iE '(avx|
avx2|avx512f|avx512_vnni|avx512bw|avx512vl|f16c|fma|sse4_2)$' |
sort -u | tr '\n' ' '
avx avx2 avx512bw avx512f avx512vl avx512_vnni f16c fma sse4_2

$ free -g | head -2
              total          used          free          shared  buff/
cache  available
Mem:           62             8           18             0
36             53

$ ldd --version | head -1
ldd (GNU libc) 2.43

$ command -v nvidia-smi vainfo
(neither found)
```

4 physical cores / 8 threads, AVX2 and AVX-512 including VNNI (the int8 dot-product extension that int8 ASR inference uses directly), 62 GiB RAM, glibc 2.43, no CUDA tooling. This is a 15–28 W laptop part: sustained all-core clock is far below the 4.7 GHz single-core boost, so throughput estimates must not be anchored to peak clock.

The box is currently saturated. Measured at 22:21 while the lumen index --force rebuild and the constitution sweep were running:

```
$ uptime
 22:21:45 up 10:22,  3 users,  load average: 11.21, 9.25, 8.29
$ ps -eo pcpu,rss,comm --sort=-pcpu | head -4
%CPU   RSS COMMAND
 385 752488 ollama
90.0 47520 ffmpeg
41.7 478044 bash
```

Load average 11.2 on 8 threads, with ollama alone consuming ~3.85 cores. **Every wall-clock figure in this document assumes an otherwise-idle machine.** Running transcription against the current load would roughly double it. Scheduling is part of the design, not an afterthought.

Tooling actually present — with two corrections

```
$ for t in ffprobe ffmpeg python3 whisper sox mediainfo pdftotext
pdfinfo pdftoppm tesseract ollama nvidia-smi; do
    p=$(command -v "$t"); [ -n "$p" ] && echo "FOUND $t -> $p" ||
echo "MISSING $t"; done
FOUND ffprobe -> <home>/bin/ffprobe
FOUND ffmpeg -> <home>/bin/ffmpeg
FOUND python3 -> /usr/bin/python3
FOUND whisper -> /usr/bin/whisper
MISSING sox
MISSING mediainfo
FOUND pdftotext -> /usr/bin/pdftotext
FOUND pdfinfo -> /usr/bin/pdfinfo
FOUND pdftoppm -> /usr/bin/pdftoppm
FOUND tesseract -> <home>/local/bin/tesseract
FOUND ollama -> /usr/bin/ollama
MISSING nvidia-smi
```

Two of those “FOUND” lines are traps. Both are corrected here because plan.md currently repeats one of them.

Correction 1 — ffprobe is not installed. The name on PATH is ffmpeg.

```
$ ffprobe -show_format -show_streams "...Recording.mp4"
Unrecognized option 'show_format'.
Error splitting the argument list: Option not found

$ ls -la <home>/bin/ffprobe <home>/bin/ffmpeg
lrwxrwxrwx ... <home>/bin/ffmpeg -> <home>/cache/ms-playwright/
ffmpeg-1011/ffmpeg-linux
```

```
lrwxrwxrwx ... <home>/bin/ffprobe -> <home>/.cache/ms-playwright/
ffmpeg-1011/ffmpeg-linux
```

Both names are symlinks to the **same** Playwright-bundled binary, which is `ffmpeg`, not `ffprobe`. It is a genuine static FFmpeg 7.0.2 (johnvansickle build) and decodes this file correctly — but it does not accept any `ffprobe` option.

`plan.md` line 27 states “ffmpeg/ffprobe 7.0.2 (present on host)”. The `ffmpeg` half is correct; **the `ffprobe` half is not**. Any pipeline step that shells out to `ffprobe` will fail on this host with an error that looks like a bad argument rather than a missing tool. **The pipeline must probe with `ffmpeg -i` (or install real `ffprobe`) and must assert at startup that `ffprobe -version` reports `ffprobe`, not `ffmpeg`.**

Also note `ffmpeg` lives in a **Playwright cache directory**.

`npx playwright install` may replace or remove it. The pipeline must not silently depend on a browser-automation cache for its media toolchain — it must resolve `ffmpeg` explicitly and fail loudly if absent.

Correction 2 — `/usr/bin/whisper` is not OpenAI Whisper. It is a microphone loopback GUI.

```
$ rpm -qi whisper | sed -n '1,3p;18,22p'
Name       : whisper
Version    : 1.3.1
Release    : alt1
URL        : https://github.com/mijorus/whisper
Summary    : Listen to your microphone
Description :
Whisper allows you to listen to your microphone through your speakers.
```

A GTK4 desktop application by Lorenzo Paderi (mijorus). It contains **no speech recognition whatsoever**. Anything that finds `whisper` on `PATH` and concludes ASR is available is wrong.

There is no working speech-recognition engine on this machine today.

0. What is actually in the recording — MEASURED

Redaction. Where this document shows the recording or the PDF by name, the filename is elided — ...Recording.mp4, ...Notes by Gemini.PDF, and Milos teaching ... AI workflows in the captured pdfinfo title — because the literal name carries a third party’s first name. The ... elides that name and nothing else. workshop/chapters/01/ holds exactly one of each, so nothing here is ambiguous. See [docs/content-boundary-incident-2026-09-01.md](#) §11.

```
$ cd workshop/chapters/01
$ ffmpeg -hide_banner -i "...Recording.mp4"
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from '...Recording.mp4':
  Metadata:
    major_brand      : isom
    minor_version    : 512
    compatible_brands: isomiso2avc1mp41
    encoder          : Google
  Duration: 01:55:28.75, start: 0.000000, bitrate: 2161 kb/s
  Stream #0:0[0x1](und): Video: h264 (High) (avc1 / 0x31637661),
yuv420p(progressive), 1920x1080, 2029 kb/s, 24 fps, 24 tbr, 12288
tbn (default)
  Stream #0:1[0x2](und): Audio: aac (LC) (mp4a / 0x6134706D),
48000 Hz, stereo, fltp, 128 kb/s (default)
```

Property	Measured value
Duration	01:55:28.75 = 6928.75 s = 115.48 min = 1.9246 h
Container	MP4 (isom/avc1), encoder: Google — a Google Meet recording
Video	H.264 High, 1920×1080, yuv420p progressive, 24 fps, 2029 kb/s
Audio	AAC-LC, 48 000 Hz, stereo, 128 kb/s
Subtitle / caption track	none — only streams 0:0 and 0:1 exist
File size	1 871 981 557 bytes (1.744 GiB) — matches video_size= in the .sha256 manifest

This duration is the denominator of every cost estimate below. The spec’s “1.8 GB” is a size, not a workload; the workload is **6928.75 seconds of audio**.

0.1 The “stereo” track is dual-mono — there is no spatial speaker cue

```
$ ffmpeg -ss 1800 -t 45 -i "...mp4" -vn -filter_complex \
    "[0:a]channelsplit=channel_layout=stereo[l][r];
    [l]volumedetect[lo];[r]volumedetect[ro]" \
    -map "[lo]" -f null - -map "[ro]" -f null -
[Parsed_volumedetect_2] mean_volume: -19.1 dB      # left
[Parsed_volumedetect_2] max_volume:  -0.2 dB
[Parsed_volumedetect_1] mean_volume: -19.1 dB      # right
[Parsed_volumedetect_1] max_volume:  -0.2 dB

$ ffmpeg -ss 1800 -t 45 -i "...mp4" -vn -af "pan=mono|c0=c0-
c1,volumedetect" -f null -
[Parsed_volumedetect_1] mean_volume: -90.3 dB
[Parsed_volumedetect_1] max_volume:  -52.5 dB
```

The **L-R difference signal sits at -90.3 dB mean / -52.5 dB peak** — that is AAC quantisation noise, not content. The two channels are identical: this is a single mono mix duplicated across two channels.

Two consequences, both load-bearing later:

1. Downmixing to mono for ASR **discards nothing**. -ac 1 is lossless with respect to content.
2. **Channel-based speaker separation is impossible**. Any speaker attribution must be acoustic (voice-embedding clustering), not spatial. See §4.

0.2 Level is uniform end to end — and heavily AGC-compressed

Ten 45-second windows spanning the whole recording, from t=0 to t=6870 (58 s before the end):

```
$ for off in 0 600 1500 2400 3300 4200 5100 6000 6600 6870; do
    ffmpeg -ss $off -t 45 -i "...mp4" -vn -af volumedetect -f null -
2>&1 | grep -E "mean_volume|max_volume"
done
```

offset (s)	mean dBFS	max dBFS
0	-18.5	-0.0

offset (s)	mean dBFS	max dBFS
600	-18.8	-0.0
1500	-18.6	-0.2
2400	-18.8	-0.1
3300	-19.2	-0.0
4200	-19.0	-0.0
5100	-19.1	-0.0
6000	-20.1	-0.0
6600	-18.3	0.0
6870	-17.8	0.0

Mean level varies by only **2.3 dB across the entire 1 h 55 m**, and peak is pinned at ~0 dBFS throughout. A 5-minute window shows **294 samples at full scale**:

```
$ ffmpeg -i sample_1800.wav -af volumedetect -f null -
mean_volume: -18.8 dB
max_volume: 0.0 dB
histogram_0db: 294
```

That is aggressive automatic gain control plus limiting — standard Google Meet behaviour. It matters for three reasons:

- **Signal is present from $t=0$ to the end.** There is no silent head, tail, or dead middle to skip. The full 6928.75 s must be processed.
- **AGC destroys the loudness cue** that diarizers lean on to separate speakers, and mild clipping distorts timbre. This degrades diarization quality — see §4.
- **Level-threshold silence detection is useless at conventional thresholds**, which the next measurement confirms.

0.3 There is speech throughout — 41 s of long gaps in 1 h 55 m, all of them locatable

A **completed** full-file scan for silences of **≥ 3 s below -35 dBFS**:

```
$ nice -n 19 ffmpeg -nostdin -vn -i "...Recording.mp4" \
    -af "silencedetect=noise=-35dB:d=3,astats=metadata=1:reset=0"
-f null -
# ran at ~14x realtime; completed the full 01:55:28.71, exit 0
$ grep -c silence_start silence.log
8
$ grep -oP 'silence_duration: \K[0-9.]+' silence.log |
```

```
awk '{s+=$1;n++} END{printf "count=%d total=%.2f s (0.3f%% of
6928.75 s)\n",n,s,100*s/6928.75}'
count=8 total=41.33 s (0.597% of 6928.75 s)
```

Correction, recorded rather than quietly fixed. While the scan was still running this research read the counter twice and saw 0, and an earlier draft of this section stated that the recording contains **zero** long silences. That was a **mid-run reading reported as a result**. The completed scan finds **8**. The claim is withdrawn and replaced with the table below. The methodological lesson is worth keeping: a progress counter is not a result, and the scan's completion (time=01:55:28.71, exit 0) is what makes the number quotable.

All eight, with their exact positions:

#	start (s)	end (s)	duration (s)	position in session
1	4552.33	4557.95	5.62	1:15:52
2	4626.42	4631.68	5.27	1:17:06
3	4872.28	4875.31	3.03	1:21:12
4	5009.90	5019.48	9.58	1:23:30 — longest
5	5025.19	5031.98	6.79	1:23:45
6	5998.99	6003.64	4.65	1:39:59
7	6063.45	6066.48	3.03	1:41:03
8	6925.35	6928.71	3.37	1:55:25 — the trailing tail

Two structural observations: seven of the eight fall between **t=4552 and t=6066** (1:15:52 → 1:41:06), a stretch with noticeably more dead air than the rest of the session; and the eighth is the recording's silent tail. **Total 41.33 s — 0.597 % of the recording.** No skippable region of any consequence exists, and every gap is individually accounted for, which is what SC-001 requires.

Whole-file astats confirms the scan covered everything and corroborates the level sampling:

```
$ grep -E "RMS level dB|Peak level dB|RMS peak dB|Number of
samples" silence.log | tail -4
RMS level dB: -18.935561
Peak level dB: 0.149787
RMS peak dB: -5.110240
Number of samples: 332578240
```


332 578 240 samples ÷ 48 000 Hz = **6928.71 s** — the full duration was processed. Whole-file RMS of **-18.94 dB** matches the ten spot samples in §0.2 (-17.8 ... -20.1), and **Peak level +0.15 dB** confirms clipping across the whole recording, not just in the sampled window.

Eight short gaps do not distinguish *continuous speech* from *continuous noise*, though. The discriminator is **short-gap structure**: conversational speech is modulated at a syllabic / inter-utterance rate, steady noise is not. Measured on a 300 s window at t=1800:

```
$ for th in -30 -40 -50; do
    ffmpeg -i sample_1800.wav -af "silencedetect=noise=${th}
dB:d=0.3" -f null -
done
threshold -30dB, min 0.3s -> 84 gaps, 53.9s total silence out of
300s
threshold -40dB, min 0.3s -> 72 gaps, 42.9s total silence out of
300s
threshold -50dB, min 0.3s -> 57 gaps, 33.9s total silence out of
300s
```

84 pauses of ≥0.3 s in 5 minutes, totalling 53.9 s (18.0 % of the window). That is the pause structure of two people talking. Combined with the long-gap result, the picture is:

Continuous conversational speech for the full 6928.75 s, roughly 82 % voice-active, broken by normal inter-utterance pauses plus 8 longer gaps totalling 41.33 s (0.6 %). No region can be skipped on silence grounds.

Those short gaps are also the natural cut points for chunking — 84 of them per 5 minutes means a valid silence-aligned boundary is always available near any target offset. See §6.

UNVERIFIED: that the modulated signal is *intelligible speech in a known language*, and the speaker count. Nobody listened to the audio, and no ASR was run — that was the explicit constraint on this research. The evidence above establishes *speech-like structure*, not content. The 5-minute calibration run in §2.5 settles it in one command. The notes PDF (§7) independently indicates two named speakers in English, but the PDF is a downstream artifact, not a measurement of the audio.

1. Decision — audio preprocessing

Decision. Extract once to 16 kHz, mono, signed 16-bit PCM WAV with ffmpeg, write it to a git-ignored work directory outside workshop/, hash it, and treat that WAV as the immutable input to every downstream stage. Never re-decode the MP4 per chunk.

Rationale.

- 16 kHz mono is the native input format of every Whisper-family model. Feeding anything else just makes the engine resample internally.
- \$0.1 proved the channels are identical, so -ac 1 is content-lossless, not a compromise.
- It is **cheap**. Measured:

```
$ /usr/bin/time -f "elapsed=%es cpu=%P" ffmpeg -ss 1800 -t 300
-i "...mp4" \
    -vn -ar 16000 -ac 1 -c:a pcm_s16le -y sample_1800.wav
elapsed=1.31s cpu=99%
$ ls -la sample_1800.wav
-rw-r--r-- ... 9600078 ... sample_1800.wav
```

300 s of audio in **1.31 s** → ~229× **realtime**, single-threaded.

DERIVED: full file $\approx 6928.75 / 229 \approx 30$ s **wall clock**, producing $6928.75 \times 16000 \times 2 = 221\,720\,000$ bytes ≈ 211 MiB. Negligible against a multi-hour ASR job, and 211 MiB is trivial against 62 GiB RAM and 1.9 TiB free on the project volume.

- Decoding once removes a whole class of resume bugs: if each chunk re-seeks into the MP4, a resumed run can land on a slightly different sample offset (AAC decoder priming, keyframe seeking) and silently shift timestamps. One decode, one hash, one offset arithmetic.
- **FR-006 (preserve source unmodified)** is satisfied structurally: the MP4 is opened read-only and the WAV is written elsewhere. workshop/.gitignore already ignores *.mp4; the work directory must be ignored too so the 211 MiB WAV is never committed.

Alternatives considered.

- *Feed the MP4 directly to the ASR engine.* Both candidate engines will do this by invoking ffmpeg internally. Rejected: it re-decodes 1.7 GiB per chunk, it hides the decode step from the resume logic, and

it makes the exact PCM the recogniser saw unhashable — which breaks the “same input $\Rightarrow$ same partition” property that resumability depends on.

- *Extract per-chunk on demand.* Rejected for the seek-offset drift reason above. The saving would be ~211 MiB of disk; the cost is timestamp integrity.
 - *Denoise / normalise / high-pass before ASR.* Rejected. Whisper is trained on unprocessed audio and extra processing typically hurts. The recording is already AGC'd (§0.2); a second gain stage would compound the clipping. **UNVERIFIED** whether targeted processing would help here — it cannot be known without a measured A/B, and §5's harness is exactly the instrument that would settle it. Deferred, not dismissed.
 - *Keep 48 kHz.* Rejected — the models downsample to 16 kHz regardless; keeping 48 kHz triples the file for no information gain above the 8 kHz Nyquist limit the models use.
-

2. Decision — speech-recognition engine

Decision. *faster-whisper* (CTranslate2 backend), CPU, int8, **model large-v3-turbo**, installed into a **project-local virtualenv** — not system-wide. `whisper.cpp` from the ALT repository is the documented fallback. **ollama is ruled out** for this job.

The model choice (`large-v3-turbo` vs `small.en`) is **provisional** and must be settled by the 5-minute calibration run in §2.5 before the full job is scheduled.

2.1 ollama — it *can* do speech-to-text, and it is still the wrong tool

The brief said to verify rather than assume. Verified — and the answer is more interesting than “no”.

The transcription route exists. Probing the running instance:

```
$ curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:11434/api/transcribe -d '{}'  
404  
$ curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:11434/v1/audio/transcriptions -d '{}'  
{"error":{"message":"failed to parse multipart form: request
```

```
Content-Type isn't multipart/form-data",...}}
HTTP 400
```

The OpenAI-compatible `/v1/audio/transcriptions` endpoint is **real** — it rejected the body for the right reason (wrong content type), not with a 404. Posting an actual WAV:

```
$ ffmpeg -f lavfi -i "sine=frequency=440:duration=1" -ar 16000 -ac
1 -c:a pcm_s16le -y probe.wav
$ curl -X POST .../v1/audio/transcriptions -F "file=@probe.wav"
{"error":{"message":"model is
required",...}} HTTP 400
$ curl -X POST .../v1/audio/transcriptions -F "file=@probe.wav" -F
"model=whisper"
{"error":{"message":"model 'whisper' not
found",...}} HTTP 404
$ curl -X POST .../v1/audio/transcriptions -F "file=@probe.wav" -F
"model=jina-embeddings-code-cpu:latest"
{"error":{"message":"\"jina-embeddings-code-cpu:latest\" does not
support chat",...}} HTTP 400
```

Full request validation. And the binary really does carry a Whisper encoder:

```
$ strings -a /usr/bin/ollama | grep -oiE 'whisper[a-z0-9_.-]*' |
sort -u
whisper
whisper_enc
whisper-enc.cpp
$ strings -a /usr/bin/ollama | grep -iE 'tmtmd_audio_preprocessor|
AudioProjector|audioTokenID|<|audio_bos|>'
mtmd_audio_preprocessor
mtmd_audio_preprocessor_whisper
AudioProjector
audioTokenID
<|audio_bos|>
```

So: **ollama 0.23.4 can do speech-to-text**. The “assume it can’t” answer would have been wrong.

But it cannot do *this* job. Three independent disqualifiers:

1. **No timestamps — this alone is fatal.** The strings above show whisper used as `mtmd_audio_preprocessor_whisper` — a Whisper **encoder** projecting audio into an LLM’s token space (`AudioProjector`, `audioTokenID`, `<|audio_bos|>`/`<|audio_eos|>`). That is the multimodal audio-in-chat path, not standalone ASR decoding. Searching the binary for the fields that carry timestamps and confidence:

```
$ strings -a /usr/bin/ollama | grep -iE 'verbose_json|
timestamp_granularit|word_timestamp|avg_logprob|no_speech'
(no matches)
$ strings -a /usr/bin/ollama | grep -iE 'response_format'
json:"response_format"
ResponseFormat json:"response_format,omitempty"
```

response_format exists; **verbose_json, timestamp_granularities, word_timestamps, avg_logprob and no_speech_prob do not appear anywhere in the binary.** No segment timestamps, no word timestamps, no per-segment confidence. That fails **FR-002** and **SC-003** outright, and leaves **FR-003** with nothing to key uncertainty off.

2. No suitable model is installed.

```
$ ollama list
NAME                                ID
SIZE      MODIFIED
jina-embeddings-code-cpu:latest    08eb8276fc69
323 MB    4 days ago
ordis/jina-embeddings-v2-base-code:latest  080d707f4f4a
323 MB    5 days ago
```

Two embedding models, 160 M parameters each. **No ASR model and no generative model at all.** The official library has none either — registry.ollama.ai/v2/library/whisper/manifests/latest returns **404**. Only third-party uploads exist (dimavz/whisper-tiny → HTTP 200, a 44 MB blob whose manifest records it was pushed from C:\Users\zatul\.ollama\models\blobs\...).

3. Supply chain. Pulling an unsigned 44 MB blob from an anonymous account to transcribe a private recording of a named third party is not a defensible choice under D1, when a reproducible alternative with a published checksum exists.

This also answers an open item in plan.md. Line 71 says “Whether any *generative* model suitable for question answering is installed has **not** been confirmed”. **Measured: none is.** Only the two embedding models above. User Story 4 has no local generative model available today.

2.2 The Python ASR stack — viable, with one correction to my own first result

My first compatibility check filtered PyPI filenames for the literal substring `linux_x86_64` and reported that **no** `cp314` wheels existed for `torch`, `ctranslate2` or `onnxruntime`. **That result was wrong and is withdrawn.** The filter was the bug: manylinux wheels are tagged `manylinux_2_28_x86_64`, which does not contain `linux_x86_64`. Re-run correctly:

```
$ python3 - <<'EOF'
import json, urllib.request, re
for pkg in ["torch","ctranslate2","faster-whisper","openai-
whisper","pyannote.audio","onnxruntime","numpy"]:
    d = json.load(urllib.request.urlopen(f"https://pypi.org/pypi/
{pkg}/json"))
    ver = d["info"]["version"]; names = [f["filename"] for f in
d["releases"][ver]]
    print(pkg, ver, sorted({m for n in names for m in
re.findall(r'-(cp3\d+|py3)-', n)}))
EOF
torch 2.13.0          ['cp310','cp311','cp312','cp313','cp314']
ctranslate2 4.8.2
['cp310','cp311','cp312','cp313','cp314','cp39']
faster-whisper 1.2.1  ['py3']
openai-whisper 20250625 []          # sdist only – builds from
source
pyannote.audio 4.0.7   ['py3']
onnxruntime 1.29.0     ['cp311','cp312','cp313','cp314']
numpy 2.5.2           ['cp312','cp313','cp314','cp315']
```

Concrete wheels confirmed present, e.g. `torch-2.13.0-cp314-cp314-manylinux_2_28_x86_64.whl`, `ctranslate2-4.8.2-cp314-cp314-manylinux_2_27..manylinux_2_28..whl`, `onnxruntime-1.29.0-cp314-cp314-manylinux_2_28_x86_64.whl`.

This host runs **Python 3.14.6** and **glibc 2.43 ≥ 2.28**, so `manylinux_2_28` wheels are installable. `venv` and `ensurepip` work, and there is no PEP-668 EXTERNALLY-MANAGED marker, so a project-local virtualenv needs no system changes:

```
$ python3 --version
Python 3.14.6
$ python3 -c "import venv, ensurepip; print('venv OK, ensurepip
OK')"
```

```
venv OK, ensurepip OK
$ ls /usr/lib64/python3*/EXTERNALLY-MANAGED /usr/lib/python3*/
```

EXTERNALLY-MANAGED
(No such file or directory)

UNVERIFIED: that these wheels actually import and run on *this* machine. Nothing was installed. cp314 support across this stack is new, and “a wheel exists on PyPI” is not “it works here”. The installation step must therefore be a **gate with a smoke test**, not an assumption — see §2.5. This is the single largest execution risk in the transcription plan, and the whole reason `whisper.cpp` is kept as a documented fallback rather than dismissed.

2.3 `whisper.cpp` — packaged for this distro, with a GPU-shaped catch

```
$ apt-cache search whisper
whisper.cpp - Port of OpenAI's Whisper model in C/C++
whisper.cpp-cpu - whisper.cpp backend for CPU
whisper.cpp-cuda - whisper.cpp backend for NVIDIA GPU
whisper.cpp-vulkan - whisper.cpp backend for GPU
whisper.cpp-ggml-base - Whisper base multilingual model in ggml
format
whisper.cpp-ggml-base.en - Whisper base English-only model in ggml
format
libwhisper1 - Shared libraries for whisper.cpp
```

```
$ rpm -q whisper.cpp whisper.cpp-cpu libwhisper1
package whisper.cpp is not installed
package whisper.cpp-cpu is not installed
package libwhisper1 is not installed
```

Version **1.9.1-alt4** is available (not installed). Attractive: one native binary, AVX-512-aware, no Python. But:

```
$ apt-cache show whisper.cpp | grep -E '^Depends' | sed 's/, /
\n      /g' | grep -i whisper
    whisper.cpp-cuda (= 1.9.1-alt4:...)
    whisper.cpp-vulkan (= 1.9.1-alt4:...)
```

The CLI package hard-depends on the CUDA *and* Vulkan backends — and notably **not** on `whisper.cpp-cpu`. Installing it pulls GPU backends onto a machine where this repository has *deliberately* excluded the GPU: `docs/setup-agents-wizard/OLLAMA-NAN-WEDGE.md` records that only `GGML_VK_VISIBLE_DEVICES=-1` yields `library=cpu`, after an i915 fault corrupted the index. The CPU-only backend is clean by comparison:

```
$ apt-cache show whisper.cpp-cpu | grep -E '^Depends' | sed 's/, /
\n/g' | grep -vE 'libc|libm|libstdc|libgcc|ld-linux|rtld'
libwhisper1 (= 1.9.1-alt4:...)
```

If the fallback is used, it must be installed with the GPU backends neutralised (GGML_VK_VISIBLE_DEVICES=-1 is already the machine-wide setting) **and** invoked with whisper.cpp's own no-GPU flag, with library=cpu asserted in the run manifest. That is extra surface the primary choice does not carry.

Only the base models are packaged; anything better comes from Hugging Face anyway. Measured sizes (`curl -sIL ... | grep content-length`):

ggml model	size
ggml-tiny.en.bin	74 MiB
ggml-base.en.bin	141 MiB
ggml-small.en.bin	465 MiB
ggml-small.en-q5_1.bin	181 MiB
ggml-medium.en.bin	1462 MiB
ggml-medium.en-q5_0.bin	514 MiB
ggml-large-v3-turbo.bin	1549 MiB
ggml-large-v3-turbo-q5_0.bin	547 MiB
ggml-large-v3.bin	2951 MiB

All fit comfortably in 62 GiB RAM and 1.9 TiB free disk.

2.4 Why faster-whisper wins

Requirement	faster-whisper (CTranslate2)	whisper.cpp 1.9.1	ollama 0.23.4
Runs CPU-only on this box	yes, int8 + AVX-512/VNNI	yes	yes
Segment timestamps (FR-002)	yes, native	yes (t0/t1)	no
Word timestamps (SC-003)	yes, word_timestamps=True	yes, but needs --dtw + matching alignment heads	no
			no

Requirement	faster-whisper (CTranslate2)	whisper.cpp 1.9.1	ollama 0.23.4
Per-segment confidence (FR-003)	avg_logprob, no_speech_prob, compression_ratio	per-token p via --output- json-full	
Built-in VAD (chunking + hallucination guard)	yes (Silero)	partial	n/a
Scriptable checkpointing (FR-029)	Python API — direct	drive the CLI per chunk	n/a
Avoids pulling GPU backends	yes	no (CUDA+Vulkan are hard deps)	yes
Needs PyTorch	no (CTranslate2 only)	no	no
Model provenance	Hugging Face, versioned + hashed	Hugging Face, versioned + hashed	anonymous blob

The decisive column is **confidence**. FR-003 requires uncertain passages to be *marked rather than guessed*, and faster-whisper is the only candidate that hands back the three signals (no_speech_prob, avg_logprob, compression_ratio) that make that mechanical rather than editorial. Everything else is recoverable; that is not.

Second: it needs **CTranslate2, not PyTorch** — a ~50 MB wheel instead of a ~900 MB one, and no GPU runtime pulled in. Third: the Python API makes the chunk/checkpoint driver of §6 a short script rather than CLI orchestration.

Alternatives considered.

- *openai-whisper (the reference implementation)*. Rejected. It is **sdist-only** on PyPI (pytags [] above) so it builds from source, it requires the full PyTorch wheel, and it is several times slower than CTranslate2 on CPU for identical output quality. It offers nothing the chosen option lacks.
- *whisper.cpp*. **Not rejected — retained as the documented fallback**, specifically because the cp314 wheel risk in §2.2 is unverified. If the venv fails to build, this is the escape hatch, at the

cost of the GPU-backend dependency and more work to extract confidence.

- *ollama*. Rejected on timestamps (§2.1). Not a close call.
- *A cloud ASR API (OpenAI, Deepgram, AssemblyAI)*. **Rejected outright, non-negotiable**. D1 makes the curriculum local/internal only, FR-024 forbids content leaving the machine, and the recording is a private session with an identifiable third party who has not consented to publication. This is a governance constraint, not a preference — it would be excluded even if it were faster and more accurate, which it would be.
- *WhisperX*. Attractive on paper — forced alignment gives much tighter word timestamps, and it bundles diarization. Rejected as the primary: it adds PyTorch, torchaudio, a separate alignment model per language, and pyannote, i.e. the entire dependency surface the chosen option avoids, and it is the least likely of all candidates to install cleanly on Python 3.14 today. Revisit only if §5 measures the timestamp error as failing SC-003.

2.5 Wall-clock estimate — and the calibration run that must happen before it is quoted as fact

No ASR benchmark was run on this machine (the brief forbade it; the box is saturated). The table below is therefore **UNVERIFIED**: it is the published CPU throughput class for these models on 4-core AVX2/AVX-512 x86 parts, applied to this recording's measured 6928.75 s.

speed factor = audio seconds processed per wall-clock second. Wall clock = 6928.75 / factor.

Model (int8, CPU, 8 threads)	Speed factor (est.)	Wall clock for 6928.75 s	Quality class
tiny.en	12–20×	6–10 min	poor — proper nouns and technical terms fail
base.en	7–12×	10–17 min	weak
small.en	2.5–4×	29–46 min	usable baseline
distil-large-v3	3–6×	19–39 min	near-large, English-only
	1.5–3×	38–77 min	near-large

Model (int8, CPU, 8 threads)	Speed factor (est.)	Wall clock for 6928.75 s	Quality class
large-v3- turbo			
medium.en	1–1.8×	64–115 min	good
large-v3	0.4–0.8×	2 h 24 m – 4 h 49 m	best

Recommended: large-v3-turbo, int8 → an estimated 40–80 minutes on an idle machine. It keeps large-v3’s encoder (where recognition quality lives) and cuts the decoder from 32 layers to 4, which is where the speed comes from. For a session dense in technical proper nouns — the notes PDF alone shows *MCP*, *Codegraph*, *Lumen*, *llama.cpp*, *SpecKit*, *Opus*, *Fable* — encoder quality is exactly what matters.

Two multipliers the estimate must not hide:

- **Contention.** At the load measured in §Machine baseline (LA 11.2, ollama at 385 % CPU), expect **roughly 2× these figures**. The job must be scheduled after the index rebuild, or niced and thread-limited, and the run manifest must record the load average at start.
- **Thermals.** A 15–28 W laptop part will throttle under a sustained 40–80 minute all-core load. The upper end of each band is the realistic planning number.

The calibration run — mandatory before the full job is scheduled

This converts the table above from UNVERIFIED to MEASURED at a cost of a few minutes, and it is the same command that resolves the “is it intelligible speech?” gap flagged in §0.3.

1. Build the venv; assert import ctranslate2, faster_whisper succeeds (settles §2.2’s risk).
2. Transcribe **exactly the 300 s window already extracted at t=1800** with the candidate model, on an otherwise-idle machine, recording wall clock, peak RSS and load average.
3. **Extrapolate:** `full_estimate = 6928.75 / (300 / elapsed_seconds)`.
4. Read the output. If it is coherent English matching the notes PDF’s topics, §0.3’s gap is closed. If it is not, stop — the problem is the audio or the model, and a 40-minute run would only produce 40 minutes of garbage.

5. Repeat for `small.en` and record both, so the quality/time trade-off is a measurement and the final model choice has evidence behind it.

Record all of it in the run manifest. **Until step 3 has been executed, the wall-clock figure quoted to the operator must carry the UNVERIFIED marker.** A 5-minute calibration is cheap; a wrong multi-hour commitment is not.

3. Decision — timestamps

Decision. Request **word-level timestamps** (`word_timestamps=True`), persist both the segment span and each word's span, and derive every published passage timestamp from the **first word's** start time rather than the segment's. Verify the result against SC-003 by measurement (\$5), not by assumption.

Rationale.

- **How they are obtained.** Whisper's decoder emits special time tokens on a 20 ms grid, giving *segment*-level timestamps for free. Word-level timestamps are produced separately, by **dynamic-time-warping alignment over the model's cross-attention weights** between audio frames and emitted tokens. The two mechanisms have very different error characteristics, which is the whole reason for this decision.
- **Accuracy in practice — UNVERIFIED here, and the failure mode matters.** Published behaviour of Whisper's segment timestamps is that they are usually good to within a second or so but drift, and — crucially — **fail in bursts, not uniformly**. The known pathologies are: a segment boundary snapping to the 30 s window edge; timestamps stalling during a repetition loop; and drift accumulating across a long run when `condition_on_previous_text` is enabled. A design that assumes uniform ± 0.5 s error will pass a casual check and then violate SC-003 exactly where the transcript is already weakest. Word-level DTW timestamps are materially tighter and, more importantly, degrade locally rather than propagating.
- **Why the first word, not the segment.** SC-003 requires a reader clicking a passage to land within **5 s of the spoken content**. Segment start times can precede the first spoken word by a

noticeable margin when the segment opens on a pause. Using the first word's start removes that systematic bias for free.

- **Three structural defences against drift**, all of which fall out of decisions made elsewhere:
 1. Chunk boundaries are placed **inside measured silence** (§6), so no chunk begins mid-word.
 2. `condition_on_previous_text=False` (§6) stops error propagating across chunk boundaries.
 3. Absolute time is computed as `chunk_start + engine_relative_time`, where `chunk_start` comes from the persisted, hashed partition — never accumulated by addition across chunks. This makes drift **bounded within a chunk** rather than unbounded across the recording.
- **It is verified, not asserted.** §5 measures timestamp error as a first-class metric — median and 95th percentile against the SC-003 5 s bound — on the same sample used for WER. If the p95 exceeds 5 s, the documented remedy is forced alignment (WhisperX, §2.4) over the existing transcript, which does not require re-running ASR.

Alternatives considered.

- *Segment timestamps only.* Simpler and cheaper, and probably sufficient for a 5 s tolerance. Rejected because the cost of word timestamps is a single flag plus a modest slowdown, while the cost of discovering a systematic bias after the transcript is published, indexed and cross-linked is a full re-run plus invalidated citations. Cheap insurance against an expensive mistake.
- *Forced alignment (WhisperX / MFA) as the primary path.* Gives the best timestamps available. Rejected as primary on dependency weight (§2.4) and kept as the documented remedy if §5 measures a failure. Adding it speculatively would be optimising against a problem not yet observed.
- *Fixed-interval timestamps interpolated across a chunk.* Rejected outright — it manufactures precision the data does not contain, which is the exact failure mode §11.4 forbids.

4. Speaker attribution

Decision. Ship US1 with speaker attribution left as an **explicitly-declared unknown**, and treat diarization as a **separately-gated, measured add-on** rather than part of the transcript's critical path. If diarization is run, its labels are stored

as machine-provenance hints and their accuracy is **measured on the same sample as WER** (§5). If the measured figure is poor, the transcript states plainly that it cannot attribute speakers.

FR-005 explicitly permits this: *“attribute passages to distinct speakers where the recording allows it, and state where it cannot.”*

Rationale — this recording is close to the worst case for diarization.

1. **No spatial cue.** §0.1 measured the L–R difference at –90.3 dB: the track is dual-mono. The single most reliable speaker cue in a two-party call — channel separation — **does not exist here**. Had the channels differed, attribution would have been near-perfect and nearly free. They do not.
2. **AGC has flattened the loudness cue.** §0.2 measured mean level varying by only 2.3 dB across 1 h 55 m with peaks pinned at 0 dBFS and 294 clipped samples per 5 minutes. Google Meet’s gain control normalises the two speakers to the same perceived level and mildly distorts timbre — removing the second cue and degrading the third (voice-embedding quality).
3. **What is left is embedding clustering alone**, on codec-compressed, AGC-flattened audio. That is precisely the condition under which speaker-embedding models are least reliable.

What diarization would cost. `pyannote.audio 4.0.7` (py3 wheel, verified available) requires **PyTorch** — the full ~900 MB cp314 wheel that §2.4 chose faster-whisper specifically to avoid — plus a Hugging Face account, an access token, and manual acceptance of gated model terms. On CPU it would add roughly the same order of wall clock again as the ASR pass itself.

UNVERIFIED: the diarization error rate that would actually be achieved on this audio. Nothing was run. Stating a DER figure here would be exactly the bluff §11.4.6 forbids. What is measured is that the two cues that make diarization easy are both absent.

Say plainly: for this recording, machine speaker attribution should be treated as **unreliable until measured**, and it must never be presented to a reader as fact on the strength of the tool having run.

Alternatives considered.

- *Human attribution.* Cheap and reliable here, and the strongest option. It is a **two-speaker teaching session** — one instructor, one

learner, with long single-speaker stretches (the notes PDF's structure suggests topic-length monologues). A human marking speaker-change points at section boundaries during the §5 review pass gets near-100 % accuracy for a small fraction of the effort of installing, running and then validating a diarization stack. **This is the recommended route.** Both speakers are already identified by name in the notes PDF, so the human corrector does not have to work them out — but that PDF is private and **the third party's name is deliberately not written into this public repository** (see the content boundary note in §8).

- *pyannote.audio as primary*. Rejected as primary for the dependency and reliability reasons above. Available as a later enhancement once US1 has shipped, gated on a measured DER.
 - *Channel-based separation*. **Rejected because it is impossible** — measured, §0.1. Recorded explicitly so nobody re-proposes it.
 - *Asserting speakers from the notes PDF*. Rejected. The PDF names the participants but carries no timing, so any mapping to passages would be invention. It is a gazetteer of names (§7), not an attribution source.
-

5. Decision — how accuracy is MEASURED, not claimed

This is the anti-bluff requirement. SC-002 demands “*a random sample of at least 30 passages*” with “*the measured figure published alongside the transcript*”; FR-004 demands the **method** be stated. A design that cannot produce a defensible number fails.

Decision. Measure **Word Error Rate** against a **blind human reference**, over **≥30 randomly sampled 30-second audio windows** stratified across the recording, with a **committed normalisation rule set**, a **seeded** sampler, and a scorer that carries its own **paired mutation proof**. Publish WER with a confidence interval, plus three companion metrics that WER alone cannot capture.

5.1 What the reference is

A human verbatim transcript of the sampled windows — and nothing else.

Rejected candidates for the reference, each for a specific reason:

- *The notes PDF*. It is a **summary**, not a transcript (§7). It contains no verbatim speech and no timings. Scoring against it would measure summarisation overlap, not recognition accuracy — and would produce a large, meaningless number.
- *A second ASR engine*. Two Whisper-family models share training data and failure modes; they agree on the same hallucinations. Agreement between them measures correlation, not correctness. This is the most tempting shortcut available and it is a bluff.
- *The machine output itself, reviewed for plausibility*. Circular. Whisper’s characteristic failure is fluent, plausible, wrong text — the exact error a plausibility review cannot see.

5.2 How sampling is done — and the bias that must be designed out

Sample fixed-length audio windows, not passages. This is the subtle and important point.

If the sample frame is “a passage the recogniser produced”, then **audio the recogniser dropped entirely can never be selected**. Whole-region deletions — a dead chunk, a failed job, a hallucination loop that swallowed 40 seconds — become structurally invisible, and the measured accuracy is biased upward exactly where the transcript is worst. Sampling the *audio timeline* instead makes deletions detectable, because the human transcribes what is actually there and the scorer counts every missing word.

Procedure:

1. Fix a seed; record it in the evidence file. The sample must be reproducible.
2. Divide $[0, 6928.75)$ into **10 equal strata** of 692.875 s each.
3. Draw **3 non-overlapping 30 s windows per stratum → 30 windows = 15 minutes of audio**. More windows raise precision; 30 is the spec’s floor.
4. Stratification guarantees the whole session is represented — a simple random draw can leave a 20-minute region unsampled by chance, and unsampled regions are where unmeasured error hides.
5. **Reconciliation with SC-002’s wording**. SC-002 says “ ≥ 30 passages”. A 30 s window of conversational speech contains several segments, so 30 windows cover **well over 30 passages**; the passages overlapping the sampled windows are enumerated and

counted in the report. The spec's floor is satisfied on its own terms *and* the deletion-blindness bias is removed.

5.3 The metric

Primary: Word Error Rate. $WER = (S + D + I) / N_{ref}$ — substitutions, deletions and insertions from a Levenshtein alignment of the normalised reference against the normalised hypothesis, over the reference word count. Published as WER **and** its complement (accuracy = 1 - WER), with a **95 % confidence interval** — 15 minutes of audio is a sample, and a point estimate quoted without an interval overstates what was measured.

Normalisation must be committed code, fixed before the first measurement is taken. This is where WER numbers get quietly fudged, so the rule set is explicit and versioned: lowercase; strip punctuation; collapse whitespace; apply a fixed number/contraction expansion table. It must **not** normalise away content words, and it must **not** be tuned after seeing results — the normaliser is committed, hashed, and its hash recorded in the report.

Three companion metrics, because WER alone hides the failures this spec cares most about:

Metric	What it catches	Spec tie
Coverage gap rate — fraction of reference words with no machine output anywhere in the window	Whole-region drops that a passage-sampled WER cannot see	SC-001
Timestamp error — machine time of first reference word - true onset , median and p95	Timestamp drift; p95 is checked against the 5 s bound	SC-003
Speaker attribution accuracy — fraction of reference words whose machine label matches the human's	Turns §4's diarization question into a number instead of a hope	FR-005

5.4 How a human reviewer executes it

1. **Run the sampler.** It emits `sample.json` (seed, the 30 window bounds, engine/model/versions and the WAV hash) plus 30 audio clips and 30 blank templates.
2. **Transcribe blind.** The reviewer types what they hear, **without seeing the machine output**. This is not a formality: showing the hypothesis first produces anchoring, the reviewer accepts plausible-but-wrong text, and the measured WER comes out too low. Blindness is what makes the number mean anything.
 - Genuinely unintelligible audio is marked with a designated token (e.g. [unintelligible]), never guessed.
 - The reviewer also marks the true onset time of the first word (from the waveform) and the speaker for each utterance — this is what feeds the timestamp and attribution metrics, and it is nearly free while they are already listening.
3. **Run the scorer.** It aligns, computes all four metrics, and writes the report to the repository's versioned evidence location (FR-040), alongside the seed, the normaliser hash, the model name and hash, the engine version, and the parameters of the run.
4. **Windows that are majority-unintelligible are reported separately and counted**, never silently dropped. Dropping hard windows inflates the score, which is the most common way an honest-looking accuracy figure becomes a lie.

Effort, stated honestly: careful verbatim transcription runs at roughly 4–8× realtime, so 15 minutes of audio is **1–2 hours of human work**. That is the real cost of a defensible number, and it should be budgeted rather than discovered.

5.5 The paired mutation proof (FR-032, SC-012)

The scorer ships with a test that constructs a synthetic reference/hypothesis pair with a **known** number of substitutions, deletions and insertions, and asserts the scorer returns **exactly** the expected WER. Mutating the scorer must make that test fail.

Per **FR-033**, the harness distinguishes three states and never collapses them: **PASS** (measured, threshold met), **FAIL** (measured, threshold missed), **ERROR / unable to verify** (the reference is missing, the sample is incomplete, or the self-check did not reproduce the known WER). An unrunnable check is never reported as a pass.

Alternatives considered.

- *Character Error Rate*. Rejected as primary — more forgiving and less interpretable for a reader of prose. Worth reporting as a secondary figure, since it is nearly free once alignment exists.
 - “*Reviewer reads five passages and confirms they look right.*” This is US1’s **Independent Test** and is a good acceptance check, but it is **not a measurement** and cannot satisfy SC-002. Both are needed; neither substitutes for the other.
 - *Scoring the whole transcript against a full human reference*. The gold standard, and unaffordable: ~2 hours of audio at 4–8× realtime is 8–16 hours of transcription. Sampling with a stated confidence interval is the defensible compromise, and the interval is what makes it honest.
-

6. Decision — resumability and progress (FR-029)

Decision. VAD-derived, silence-aligned chunks of ≤ 300 s, each transcribed independently with `condition_on_previous_text=False`, each written to its own JSON file by **write-temp** → **fsync** → **atomic rename**. Resume by recomputing the partition deterministically and skipping chunks whose output exists and whose recorded input hash matches. Coverage is then **provable by construction**, not by inspection.

Rationale.

The pipeline.

1. **Decode once** → `audio.wav` (16 kHz mono, 211 MiB, ~30 s — measured §1). Hash it.
2. **Run VAD once** over the whole WAV → `vad.json`, the speech/non-speech region list, stamped with the WAV hash. §0.3 measured ~18 % non-speech in short pauses, so this materially reduces audio actually fed to the decoder *and* suppresses Whisper’s characteristic hallucination over non-speech.
3. **Partition deterministically** → `chunks.json`. Accumulate speech regions up to ≤ 300 s, and **always cut inside a silence gap**, never inside a speech region. §0.3 measured 84 gaps ≥ 0.3 s per 5 minutes, so a valid cut point is always available near any target boundary. Each entry records index, absolute start/end, and a content hash.

4. **Transcribe chunk i** → write chunks/NNN.json.tmp, fsync, rename() to chunks/NNN.json. **The atomic rename is the checkpoint.** POSIX rename within a filesystem is atomic, so a crash at any instant leaves each chunk file either absent or complete — never half-written. A half-written chunk that parsed as valid JSON would be the worst outcome: silent, plausible data loss.
5. **Resume:** recompute steps 2–3 (same WAV hash ⇒ identical partition), then skip any chunk whose output exists **and** whose embedded input hash matches. Everything else is redone. No state beyond the files themselves; nothing to corrupt.
6. **Assemble:** offset each chunk's engine-relative times by its absolute chunk_start.

Why 300 s. It bounds re-work on interruption to ≤ 5 minutes (≤ 3 min of compute at the recommended model's estimated rate), while keeping per-chunk model-invocation overhead negligible. $6928.75 \text{ s} \div 300 \text{ s} \approx \mathbf{24 \text{ chunks}}$, so progress is reported at $\sim 4\%$ granularity — fine enough to be informative, coarse enough that checkpoint I/O is free.

How it interacts with timestamp continuity — the part that is easy to get wrong.

- Cutting **inside silence** means no word straddles a boundary, so no de-duplication or overlap stitching is needed. Overlap-and-merge is the usual approach here and it is a reliable source of duplicated and dropped words at the seams; cutting at measured silence avoids the problem instead of managing it.
- `condition_on_previous_text=False` is required for chunk independence — and it is **independently the right setting for long-form robustness**, because carrying decoder context across a long run is the primary trigger for Whisper's repetition-loop hallucinations. The setting that makes the job resumable is the same setting that makes it more accurate. That alignment is why this design is cheap.
- Absolute time is always `chunk_start + relative`, read from the hashed partition — **never** accumulated by summing chunk durations. Drift therefore cannot compound across the recording.

Coverage becomes a mechanical proof, which is what SC-001 actually needs.

The union of all passage spans and all VAD-declared non-speech spans **must equal** `[0, 6928.75)` **exactly**. Any second belonging to neither is an **unexplained gap** and fails the build.

This turns “*covering the entire recording with no unexplained gaps*” from a claim into an assertion a machine evaluates. Non-speech regions are explicitly recorded as accounted-for, so they are explained rather than merely absent — which is precisely the distinction SC-001 draws. Its paired mutation proof (FR-032): delete one chunk’s output and assert the coverage check FAILs.

The eight long gaps measured in §0.3 are a ready-made test fixture.

Their positions are known exactly (4552.33, 4626.42, 4872.28, 5009.90, 5025.19, 5998.99, 6063.45, 6925.35 s). The VAD must independently classify all eight as non-speech; if it does not, the VAD configuration is wrong and the coverage proof is resting on a broken detector. That check costs nothing and is worth wiring in before the first full run. Note that gap 8 is the recording’s **trailing tail** (6925.35 → 6928.71) — the assembled transcript must terminate with an accounted-for non-speech region, not with a passage, and an off-by-one at the tail is the most likely place for the coverage assertion itself to be wrong.

Progress reporting. Chunks completed / 24, audio-seconds completed / 6928.75, elapsed, and a projected finish from the measured rate so far — not from the §2.5 estimate. Written to the run manifest as it goes, so an interrupted run leaves a readable record of how far it got.

Determinism. Beam size, temperature schedule and seed are fixed and recorded, along with engine version, model name and model file hash. Without this, a resumed run silently mixes outputs from two configurations and the accuracy figure describes neither.

Alternatives considered.

- *Fixed-duration chunks at arbitrary offsets.* Simpler to compute. Rejected: cuts land mid-word, which forces overlap-and-deduplicate, which is where seam defects come from.
 - *One process, in-memory, no chunking.* Rejected by FR-029 directly — a 40–80 minute job that loses everything on interruption. It also holds the whole result in memory, working against the 60 % RAM cap.
 - *A database or a job queue for checkpoint state.* Rejected as unnecessary machinery for 24 work units. Files plus atomic rename have fewer failure modes than any process holding a lock, and they are directly inspectable when something goes wrong.
 - *Checkpoint every segment rather than every chunk.* Finer resume granularity, thousands of tiny files, and no way to make a partially-written chunk’s segment set coherent. Rejected — the chunk is the natural transaction boundary.
-

7. Decision — uncertainty marking (FR-003)

Decision. Persist the engine's three per-segment confidence signals plus per-word probability, map them to three rendered states — **plain text**, **word-level uncertain**, and **[inaudible]** — and **calibrate the thresholds against the \$5 human sample** rather than adopting defaults. Never emit text for a segment classified as non-speech.

Rationale.

faster-whisper returns, per segment, `avg_logprob` (mean token log-probability), `no_speech_prob`, and `compression_ratio`; and, with `word_timestamps=True`, a probability per word. All four are persisted in the immutable machine layer (FR-038) whether or not they are rendered — discarding them would make later recalibration impossible without re-running ASR.

Mapping to the spec's requirement:

Signal	What it detects	Rendered as
High <code>no_speech_prob</code> and low <code>avg_logprob</code>	Decoder emitting text over non-speech — a hallucination	[inaudible] with the time span. No text is emitted.
<code>compression_ratio</code> above threshold	Repetition loop ("okay okay okay okay...") — a classic long-form Whisper failure	Segment flagged unreliable; text retained but marked
Low per-word probability	Localised low confidence, typically proper nouns and technical terms	Word marked uncertain in the rendered markdown
Otherwise	—	Plain text

Two points that make this honest rather than decorative:

- 1. Thresholds are calibrated, not guessed.** Published defaults (`no_speech_prob > 0.6`, `avg_logprob < -1.0`, `compression_ratio > 2.4`) are a *starting point*, not an answer for this audio. Using the \$5 sample, bucket the machine words by confidence and compute the **actual** error rate per bucket; choose the threshold where marking starts separating right from wrong. The calibration curve is published with the accuracy report. Thresholds adopted without

this are themselves a bluff — they assert a confidence semantics nobody checked.

2. **State the limitation in the documentation (FR-031).** Whisper's worst failure mode is being **confidently wrong**: hallucinated text over non-speech often carries *high* token probabilities. Confidence marking therefore reduces, but does not eliminate, silent error. The user guide must say so. Claiming that unmarked text is verified text would be precisely the overreach §11.4 forbids, and it is the natural thing for this feature to accidentally imply.

Provenance (FR-038) rides along. Every passage records whether its text is machine-produced or human-corrected. The machine layer is immutable — it is the evidence of what the recogniser actually produced, and overwriting it would destroy the ability to re-measure accuracy later or to recalibrate the thresholds above.

Alternatives considered.

- *Suppress low-confidence text entirely.* Rejected — FR-003 requires unclear passages to be **marked**, not omitted; deletion creates the unexplained gaps SC-001 forbids and destroys information a human corrector could use.
- *Emit best-guess text with no marking.* Rejected — this is the exact “silently guessed at” behaviour FR-003 prohibits.
- *A second model as a confidence cross-check (agreement ⇒ confident).* Rejected for the same reason as in §5.1: Whisper-family models share failure modes and agree on their hallucinations. It would produce a confidence signal that is confidently wrong in precisely the places that matter, at double the compute.

8. Decision — the notes PDF

Decision. Extract the PDF's text layer with `pdftotext`, store it as a **separate supporting material** with `provenance = notes-pdf`, and use it for **structure, proper-noun correction and coverage cross-checking only**. **Never** merge its text into the transcript, and **never** use it as accuracy ground truth. `tesseract` is **not required** — measured.

Measured facts about the PDF.

```
$ pdftinfo "...Notes by Gemini.PDF"
```

```
Title:           Milos teaching ... AI workflows - 2026/08/27 09:57
CEST - Notes by Gemini
```

[REDACTED 2026-09-09: a device/user-agent fingerprint and an exact document creation timestamp, both extracted from a PRIVATE session's notes PDF, stood here. They identified a physical device and are withdrawn. CONTAINMENT ONLY – the values remain in this PUBLIC repository's history at commit 16cd4ba, and history rewriting is forbidden by §11.4.113. Whose device it was is UNDETERMINED.]

Producer: Skia/PDF m150

Pages: 8
Tagged: yes
File size: 417800 bytes
PDF version: 1.4

```
$ pdftotext "...Notes by Gemini.PDF" - | wc -l -w -c
389    3034    21291
```

Every page has a real text layer — so OCR is unnecessary:

```
$ for p in $(seq 1 8); do
    printf "page %s: %s chars\n" "$p" "$(pdftotext -f $p -l $p "...
PDF" - | tr -d '[:space:]' | wc -c)"; done
page 1: 2405 chars    page 2: 2597 chars    page 3: 1840 chars
page 4: 2986 chars
page 5: 3382 chars    page 6: 2058 chars    page 7:  926 chars
page 8: 2026 chars
```

And there are no diagrams to OCR — the only images are the repeated running-header banner and a small icon:

```
$ pdfimages -list "...Notes by Gemini.PDF" | head -5
page  num  type  width height color comp bpc  enc interp  object
ID x-ppi y-ppi size ratio
   1    0 image   2253   188  gray    1   8  jpeg   no
11  0    319   319 6020B 1.4%
   1    1 image    36   156  icc     3   8  image  no
17  0   102   102 2888B 17%
   1    2 smask    36   156  gray    1   8  image  no
17  0   102   102 269B 4.8%
   2    3 image   2253   188  gray    1   8  jpeg   no
41  0    319   319 6020B 1.4%
```

The 2253×188 grey JPEG repeats identically on every page — it is the title bar, not content.

tesseract is present on this host but is NOT needed for this artifact. The repository toolchain lists it; this PDF does not require it. Running OCR here would add a lossy pass over a perfect text layer.

What the PDF contains. A structured summary organised under **seven topic headings**, each with bullet points beneath it. The headings span the session's technical ground and its business ground. **The heading strings themselves are not reproduced here** — they are private content; see the content boundary note at the end of this section. It names both participants: the operator, and a third party whose name is likewise **not reproduced here**. **It contains no verbatim speech and no timestamps whatsoever.**

Three legitimate uses.

1. **Section-structure prior (FR-011).** Its headings are a strong candidate outline for dividing the transcript into navigable sections — proposed by the PDF, then **confirmed against transcript content and assigned real timestamps from the transcript**, never from the PDF.
2. **Proper-noun gazetteer.** ASR reliably mangles technical proper nouns, and the PDF supplies the vocabulary: *MCP*, *Codegraph*, *Lumen*, *llama.cpp*, *Helix QA*, *Opus*, *Fable*, *RAG*. These feed a **review checklist for a human corrector** — a list of terms to check — and optionally an ASR initial prompt. They must **not** be auto-substituted into machine output; that would manufacture text the recogniser did not produce and corrupt the immutable layer (FR-038).
3. **Coverage cross-check.** Every PDF topic should map to some transcript region. One that maps to nothing is a flag: either a transcription gap, or a topic the notes invented. Either way it is worth surfacing — which is what the spec's edge case asks for.

Why the recording is authoritative — with evidence, not just because the spec says so.

The PDF is a **Gemini-generated summary of the session**, produced by a model with its own error rate, and it demonstrably contains errors. **Reproduce the finding rather than reading it here** — the command below prints private text, so it is given as a recipe to run against the private source, not as captured output:

```
$ pdftotext "workshop/chapters/01/... - Notes by Gemini.PDF" - \
  | grep -noIE 'spatkit|speckit|llama ?cpp|codegraph|lumen|helix
qa|opus|fable'
# output not reproduced here – private content; see the content
boundary note below
```

Two mis-renderings of spoken proper nouns show up in that output, and both are checkable: the spec-driven planning toolchain's name comes out one letter wrong (this repository contains `.specify/` and

submodules/superspec, so the correct spelling is **SpecKit**), and `llama.cpp` comes out spaced and capitalised as if it were two English words. These are **concrete, checkable evidence** that the notes contain transcription-class errors, which is exactly why the spec makes the recording authoritative. A disagreement between the two is **surfaced**, never silently resolved.

Alternatives considered.

- *Use the PDF as the accuracy reference.* Rejected — see §5.1. It is a summary; scoring against it measures the wrong thing entirely.
- *Merge PDF bullets into the transcript to fill gaps.* Rejected outright. It would insert text nobody said at timestamps that do not exist — fabrication, violating FR-003, FR-038 and §11.4.
- *OCR the PDF with tesseract.* Rejected — measured unnecessary (every page has a text layer, no diagrams). It would substitute a lossy pass for a lossless one.
- *Discard the PDF.* Rejected — it is genuinely useful for structure, vocabulary and cross-checking, and it is the only independent artifact about this session that exists.

Content boundary — read before quoting the PDF anywhere

`workshop/chapters/01/...` - Notes by Gemini.PDF lives in the **private** workshop submodule. **This repository is public.** Three classes of material from it must never be written into this tree: **verbatim text**, the **section-heading list**, and the **third party's name**. Describe the substance, cite the path, reproduce nothing. The same rule will govern the transcript once it exists — a transcript of a private recording is private until the operator says otherwise — so the §5 WER sampling procedure must report **scores**, and must not paste the sampled passages, the human reference or the machine hypothesis into this repository.

`bash scripts/verify-content-boundary.sh` enforces this, with a caveat worth knowing: its match window is **eight normalised words**, so a short heading or a five-word phrase can pass it while still being a verbatim reproduction. A green gate is a floor, not a licence.

An earlier revision of this file and of docs/workshop-curriculum/RECON.md violated all three classes. The disclosure, and the exposure that redaction does **not** undo, are recorded in [docs/content-boundary-incident-2026-09-01.md](#).

9. What must be installed (nothing was)

Reported, not performed. Nothing here was executed.

Primary path — project-local virtualenv, no system changes:

Component	Source	Notes
faster-whisper 1.2.1	PyPI (py3-none-any)	verified present on PyPI
ctranslate2 4.8.2	PyPI (cp314 manylinux_2_28)	glibc 2.43 ≥ 2.28 ✓
onnxruntime 1.29.0	PyPI (cp314 manylinux_2_28)	needed for the Silero VAD
large-v3-turbo (CT2 int8)	Hugging Face	~547 MiB–1.5 GiB depending on quantisation
real ffprobe or an ffmpeg -i probe path	—	see Correction 1 — do not assume ffprobe exists

Fallback path — distro packages (requires operator authorisation): whisper.cpp + whisper.cpp-cpu + libwhisper1 (1.9.1-alt4) — **note the CUDA/Vulkan hard dependency** documented in §2.3, and the GGML_VK_VISIBLE_DEVICES=-1 mitigation.

Only if diarization is pursued (§4): pyannote.audio 4.0.7 + torch 2.13.0 (~900 MB) + a Hugging Face token + manual acceptance of gated model terms.

Not needed: tesseract (§8), sox, mediainfo, any CUDA runtime.

10. Corrections this research forces on plan.md

plan.md says	Measured reality
line 27: “ffmpeg/ffprobe 7.0.2 (present on host)”	ffmpeg yes; ffprobe no — the PATH entry is a symlink to the Playwright ffmpeg binary and rejects ffprobe options. It is also inside an npx-managed cache.
line 71: “Whether any <i>generative</i> model ... has not been confirmed”	Confirmed: none. ollama list shows only two 160 M embedding models. US4 has no local generative model today.
line 82: “Speech-recognition engine selection and realistic wall-clock cost for a 1.8 GB recording”	The workload is 6928.75 s of audio , not 1.8 GB. Recommended engine and model in §2; wall clock estimated 40–80 min on an idle box and still UNVERIFIED pending §2.5’s calibration.
(implicit) ASR tooling exists because whisper is on PATH	/usr/bin/whisper is a microphone loopback GUI. No ASR engine is installed.

Already satisfied — no work needed. FR-007 (reassemble split recordings, verify integrity, fail loudly) is **already implemented** by workshop/scripts/extract-videos.sh, which hash-verifies each part, then the reassembled archive, then the extracted video, against the manifest:

```
$ head -5 "...Recording.mp4.sha256"
# video-archive-manifest v1
video_sha256=345a74398a440e2df3a8cacf8422e7712c3e5bab19147594811b4
video_size=1871981557
archive_sha256=67ee3b723859f1dd492c77c94d71bc5d37d80af194de6cfdbb0
part_size=50m
```

video_size matches the measured file size exactly. The transcription pipeline should **invoke** this script rather than reimplement reassembly.

11. UNVERIFIED register

Everything this research could not observe on this machine, with the reason and the command that would settle it. Per §11.4.6 none of these may be reported as established.

#	Claim	Why unverified	How to settle it
U1	The cp314 wheels (ctranslate2, onnxruntime) install and run here	Nothing was installed	Build the venv; import ctranlate2, faster_whisper. Highest execution risk in this plan
U2	ASR throughput on this CPU (the 40–80 min figure)	Benchmarking was out of scope and the box is saturated	§2.5 calibration on the existing 300 s sample
U3	The audio is intelligible speech in a known language	Nobody listened; no ASR was run. §0.3 established speech-like modulation only	§2.5 calibration output — read it
U4	Achievable WER on this recording	No transcript exists	§5, after the first full run
U5	Timestamp error against SC-003's 5 s bound	Same	§5's timestamp metric
U6	Diarization error rate	pyannote not installed; §4 argues conditions are poor	§5's attribution metric, if diarization is run
U7	Confidence thresholds	Requires the §5 sample to	§7's calibration curve

#	Claim	Why unverified	How to settle it
	appropriate for this audio	calibrate against	
U8	Speaker count is exactly two	Inferred from the notes PDF, which is a downstream artifact	Human review during \$5
U9	whisper.cpp installs without activating a GPU backend	Not installed; Depends on CUDA+Vulkan is measured, the mitigation is not	Only if the fallback is taken; assert library=cpu

12. Summary of decisions

#	Decision	Confidence
1	Decode once to 16 kHz mono PCM WAV (~30 s, 211 MiB); source untouched	MEASURED
2	faster-whisper / CTranslate2, CPU int8, large-v3-turbo, project-local venv	Reasoned; U1/U2 open
2b	whisper.cpp 1.9.1 as documented fallback; ollama ruled out — no timestamps	MEASURED
3	Word-level timestamps; publish first-word time; verify against SC-003	Reasoned; U5 open
4	Speaker attribution by human review ; diarization only as a measured add-on	MEASURED that both easy cues are absent

#	Decision	Confidence
5	WER vs blind human reference , ≥30 stratified 30 s windows , seeded, mutation-proofed	Method fixed
6	≤300 s silence- aligned chunks, atomic-rename checkpoints, coverage proved by construction	Design fixed
7	Three-state uncertainty marking, thresholds calibrated against the \$5 sample	Design fixed; U7 open
8	PDF via pdftotext for structure/ vocabulary/cross- check only; no OCR needed	MEASURED

The one thing to do next: run the \$2.5 calibration. It closes U1, U2 and U3 in a few minutes, and it is the only cheap way to avoid committing to a multi-hour job on unverified assumptions.